

In the final report, you should (1) demonstrate your attack strategies; (2) compare the attack effect for different strategies; and (3) discuss which attack strategy works best in a given scenario.

Scenario 1: Simple Average

As this scenario was the simplest, the attack strategy I went with was also the simplest. I merely took my 20 malicious ID's and had them all rate product 1 with a 1. This took product 1's average rating from 3.982905982905983 to 3.5474452554744524. Overall this didn't impact the score that much due to there being a large amount of pre-existing good reviews. As for doing anything else, I knew it wouldn't be as effective as this so I didn't try anything more. For a simple average, this is the best technique. Realistically, an attacker's best hope for this strategy would be to attack products that haven't accrued a significant amount of reviews and hope that seeing the bad ratings will turn customers away from the product.

Scenario 2a: Fixed Sliding Window

- Note: As Scenario 2 is split into two separate problems, so too will their sections be split in this final report.

Fixed Window Ending Averages:

Window 1: 4.133333333

Window 2: 4.318181818

Window 3: 3.884615385

Window 4: 3.6875

Window 5: 3.885714286

Fixed window post concentrated attack averages: (aka 20 bad-mouthing attacks done in one window, on average seems to be around a 1.5 decrease in rating)

Window 1: 2.342857142857143

Window 2: 2.738095238095238

Window 3: 2.630434782608696

Window 4: 2.1944444444444446

Window 5: 2.8363636363636364

For this one, I just decided to try running a bad mouthing attack with all 20 accounts on each of these windows. Of course in a real world situation you'd likely only be able to run one of these attacks on the window and then leave the rest as is, but even then this does prove to be the best way to tank the rating of the product for that time.

Fixed window post split attack averages: (aka 4 bad mouthing attacks split over each window, on average seems to be a .5 decrease in the rating)

Window 1: 3.473684210526316

Window 2: 3.8076923076923075

Window 3: 3.5

Window 4: 3.15

Window 5: 3.58974358974359

I also tried a version of the attack where I split my bad mouthing attacks evenly over the 150 day period. While the ratings decrease isn't as prominent, it still decreases the rating a noticeable amount. Showing that due to the windows having less total ratings, bad mouthing attacks can actually be more effective at changing the overall rating compared to a total average system.

Scenario 2b: Dynamic Sliding Window

Average reputations per different window:

0-30: 4.125

10-40: 4.285714285714286

20-50: 4.476190476190476

30-60: 4.333333333333333

40-70: 4.241379310344827

50-80: 3.909090909090909

60-90: 3.8214285714285716

70-100: 3.6551724137931036

80-110: 3.75

90-120: 3.6470588235294117

100-130: 3.95
110-140: 3.942857142857143
120-150: 3.923076923076923

Reputations post all-in attacks: (aka 20 bad mouthing attacks at the start of every cycle, resulting in around a 1-2 point decrease in scores depending on review quantity)

0-30: 2.388888888888889
10-40: 2.3529411764705883
20-50: 2.7804878048780486
30-60: 2.8181818181818183
40-70: 2.9183673469387754
50-80: 2.5238095238095237
60-90: 2.6458333333333335
70-100: 2.5714285714285716
80-110: 2.6041666666666665
90-120: 2.2162162162162162
100-130: 2.475
110-140: 2.8727272727272726
120-150: 2.9322033898305087

Much like in 2a, I first tested the strategy of using all 20 bad mouthing attacks right away, and indeed, this did perform according to expectations by lowering the scores by the largest amount. However, while this attack is the most effective initially, you quickly experience diminishing returns when you realize that by attacking at the beginning of the cycle, your attack only works for 10 days before being forgotten by the system. Theoretically it would be much better if you attacked at the end of a cycle instead of the beginning, so the score would last for 30 days instead of just 10.

Reputations post split attacking (sending 5 bad mouthing attacks at the end of a cycle over the 150 day period):

Sending on day 30:

0-30: 3.380952380952381 (Only taking effect on day 30)
10-40: 3.4210526315789473
20-50: 3.8076923076923075

Sending on day 60:

30-60: 3.7586206896551726 (Only taking effect on day 60)
40-70: 3.764705882352941
50-80: 3.3703703703703702

Sending on day 90:

60-90: 3.393939393939394 (Only taking effect on day 90)
70-100: 3.264705882352941

80-110: 3.3333333333333335

Sending on day 120:

90-120: 3.0454545454545454 (Only on day 120)

100-130: 3.36

110-140: 3.575

120-150: 3.590909090909091

So, using the new method of sending at the end of each window period, these are the results (not including possible overlap by the system possibly accounting for two “sending periods” [ex: day 30 and day 60 bad mouthing attacks being counted together]). Using this new method, you can benefit from decreasing the scores by a non-insignificant amount (sometimes even by a full point) due to the window method causing the total score pool to be smaller, and thus attacks weigh more towards the average otherwise.

In summary, for a short term severe ratings decrease, using all your attacks in a short period is the best way of getting the rating down. However for a long term decrease, spreading out your resources when attacking for maximum benefit will give you the best overall results.

Scenario 3: Forgetting Model

Beginning value of product 1: 4 -> 0.8947546959145896

Most recent value of product 1: 4 -> 3.96

Sum total of reputation score: 221.83508941541626

- Leaving the sum total because trying to calculate the average gives a score of 1.8960264052599687, which doesn't really make sense considering how well the product is actually doing.

Since forgetting the values decreases them by $.99^{\text{(however many days they were last posted since the present date)}}$ this leaves problems when trying to calculate the reputation score.

Leaving the system as is, the reputation score will continue to be overly inflated, and trying to average the sum total won't work because it will make the score of the product lower than it's supposed to be.

Overall a forgetting factor of .99 is too lenient for the forgetting algorithm to be an accurate judge of reputation. I found that a forgetting factor of .7 would give a reputation sum within the 1-5 scale, but further testing would likely be required on that.

Either way, the way the algorithm works, there are two “best” ways of using a bad mouthing attack.

The first method is using all bad mouthing attacks at once to make the rating low for a relatively shorter period.

The second is to spread out your attacks to “push out” legitimate reviews to try and keep your bad mouthing attacks most recent. The easiest way to do this would be to spread out your attacks over the 150 day period but even then this only gives good results when the forgetting factor is more severe.

Which method is better depends on how strict the forgetting algorithm is. However for a forgetting factor of .99, using all your bad mouthing attacks at once should be fine considering that the algorithm will still remember those attacks even after 150 days (though only at around 20 percent of their original relevance). As is, the current system is just a slightly improved version of the simple average (and only slightly because the program will eventually forget bad/good values sometime before the heat death of the universe, in the short term it is basically equivalent to simple average).

Scenario 4: Beta Reputation Model

$q=0.05$

Average value after product 1's values were filtered: 4.035087719

- Overall a slight increase when compared to the original simple average.

The lower and upper bounds were determined to be:

Lower Bound: 3.0

Upper Bound: 5.0

Attacking:

For this problem, I tried attacking at both the beginning and the end of the algorithm. As you can tell, both attempts yielded the same result.

Beginning attack: 3.547445255

Ending attack: 3.547445255

Lower Bound: 1.0

Upper Bound: 5.0

Overall, while the average value was higher, it seems that the relatively small q meant that the algorithm didn't filter out any bad mouthing attacks. You can tell this by the resulting attack value being literally the same as the post attack value for the simple average. For this type of rating system, the q should be higher so the lower bound will catch any obvious bad mouthing attacks. Through some trial and error, I found that a q of 0.20 prevented the simple bad mouthing attack of flooding the product with 1's.

Attacking with only 1's with a q of 0.20: 4.035087719

- As you can see, no change from the initial average.

So, to beat this system I then had my malicious ID's give the product neutral reviews of 3, thus causing the score to decrease as the algorithm would get rid of higher reviews as it receives more that are within the lower bound.

Attacking with only 3's: 3.880597015

- Also yes I did try mixing in 1's and 2's with the 3's, but I wasn't able to get the algorithm to accept those and thus attacking with only 3 was the best option to lower the review.

In summary:

If q is too lenient, using a beta reputation model seems to be only as good as having a simple average (though with a larger scale [example being 1-100] a more lenient q would probably have worked). However if q is stricter, it proves highly effective at preventing deviations in score through bad mouthing attacks. It's not that it cannot be done, but it must be done slower and can only cause the product to regress towards its lower bound.

Scenario 5: Beta Reputation Model with Reviewer Trust

Piggybacking off a lot of the work already done for Scenario 4, I was able to get the weighted average in much the same way. In this case the weighted average turned out to be:

Beta Reputation Score (Product 1 Only): 3.982905982905983

However after doing this I realized that for calculating reputation, I need to be taking into account all reviews done on all products and then using that to update the beta reputation score for product 1. Once that was done the value was the following:

Beta Reputation Score (including all values for filtering): 4.305555555555555

First attempt, the brute force attack:

Post Attack Beta Reputation Score (Product 1 Only): 1.8174757281553398

Post Attack Beta Reputation Score (including all values for filtering): 4.305555555555555

In the first case the attack worked due to the reputation system not being calibrated properly. However, once calibrated on all data, this proved to be the most effective at stopping the brute force bad mouthing attack. Of course, none of these accounts had a reputation before trying to brute force, so let's now gather reputation before attacking.

Post Attack Beta Reputation Score (post reputation gathering): 2.5657894736842106

As you can clearly tell, there is a significant change in the rating now. This was done by giving two ratings of 5 to two separate products before then proceeding to give product 1 a rating of 1. It then follows that the theoretical strongest attack against product 1 is to give good ratings to every other product on the site, but that would be unrealistic to test. Not to mention that the data

is clear, when working with this sort of reputation system, it is best to gather reputation on your malicious accounts before attacking.

- On the other hand I could have simulated a hacking by taking trusted user id's for other products and using those id's to rate product 1 a 1, but that would be a similar result as what was shown here, and thus not worth making a separate attack file for.